DAY 4

Advanced Copilot Tooling

CLI as an OS | MCP | Skills | Playwright | Computer Use

From tools inside the IDE to an AI-powered operating system

DAY 3 RECAP

What We Proved Yesterday

One paragraph → User stories → Sprint plan → Backend API → Angular frontend → Tests → Docs

Traditional estimate: ~40 person-days. You did it in under 3 hour. That's 100x.

But speed without structure is chaos. Today we learn the structure.

Today's question:

How do you make AI-powered development repeatable, auditable, and scalable?

RECAP EXERCISE

Onboard the Banking API — Properly

Before new content, let’s do something you’ll use for the rest of this workshop and beyond:

1. **Create AGENT.md for the Banking API**
   Tell all AI agents how to work with this project
2. **Create copilot-instructions.md**
   Onboard Copilot so it never asks the same question twice
3. **Ask: “How can I contribute?”**
   Get a personalized answer based on your actual role

AGENT.MD

The Universal Playbook for AI Agents

AGENT.md is a file at the root of your repo that any AI agent reads before starting work.

What goes in AGENT.md:

- Project overview and purpose
- Tech stack and architecture
- Build, test, and run commands
- Key file locations and structure
- Coding conventions and patterns
- Common pitfalls and workarounds
- Validation and CI/CD steps
- What to always/never do

Who reads AGENT.md:

GitHub Copilot	Agent mode, coding agent, CLI
Claude Code	Reads AGENT.md + CLAUDE.md
OpenAI Codex	Reads AGENT.md as primary context
Cursor / Windsurf	Follows agent instructions
Any future AI agent	AGENT.md is becoming the standard

One file. Every AI agent. Consistent behavior.

COPILOT-INSTRUCTIONS.MD

The Onboarding Document for Copilot

This isn’t the simple version from Day 2. This is production-grade onboarding.

The goal:

- Reduce rejected PRs from coding agent
- Minimize build failures and command errors
- Eliminate repeated exploration (grep, find, search)
- Agent trusts the instructions, doesn’t re-discover every time

What it must cover:

- Repository summary
- Build, test, lint, run commands (validated, in order)
- Project layout with key file locations
- Known errors, workarounds, and gotchas

COMPARISON

AGENT.md vs copilot-instructions.md

	AGENT.md	copilot-instructions.md
Location	Project root	.github/ folder
Read by	All AI agents (Copilot, Claude, Codex, Cursor)	GitHub Copilot only
Purpose	Universal project context for any AI tool	Copilot-specific coding guidelines and chat behavior
Scope	Root or subfolders (scoped rules)	Entire workspace (all chat requests)
Best for	Multi-agent teams, CI/CD agents, coding agent PRs	Daily IDE usage, chat quality, naming conventions
Recommendation	**Use both. They complement each other.**	

THE BIG REFRAME

Copilot CLI is Not a Tool

It's an Operating System

It reads files. Writes code. Runs commands. Manages git.
Orchestrates tools. Self-corrects. Deploys.
All from natural language.

The terminal is no longer where you type commands. It's where you have conversations with your machine.

SIDE BY SIDE

Traditional OS vs Copilot CLI

TRADITIONAL OS

File Manager → browse files

Text Editor → write code

Terminal → run commands

Browser → test endpoints

Git client → manage versions

Package manager → install deps

CI dashboard → check builds

→

COPILOT CLI

"Show me the project structure"

"Create a new service class"

"Build and run the tests"

"Hit the transfer endpoint"

"Commit with a good message"

"Add this dependency"

"Generate a CI pipeline"

Same capabilities. Natural language interface. One tool.

CLI IN ACTION

What the OS Can Do

Explain

Understand any command, script, or error message in plain English

"Explain what mvn clean install -DskipTests does"

Generate

Create scripts, configs, Dockerfiles, CI pipelines from description

"Generate a GitHub Actions workflow for this Spring Boot app"

Multi-Step

Plan and execute complex workflows: build, test, fix, commit, push

"Build this project, run tests, fix any failures, then commit"

Background

Delegate tasks to run autonomously while you keep working

"In the background, refactor the service layer and open a PR"

MCP SERVERS

Extending the OS with External Tools

MCP (Model Context Protocol) connects Copilot to external services, APIs, and data sources.

Think of MCP as USB ports for AI. Plug in a server, Copilot gains new capabilities.

Database MCP

Query your database directly from chat

GitHub MCP

Create issues, PRs, manage repos from prompts

Browser MCP

Navigate websites, scrape data, test UIs

Slack/Jira MCP

Read tickets, post updates, search messages

SKILLS

Teaching the OS New Tricks

Skills are packaged bundles of capabilities — tools, instructions, hooks, and MCP configs — that you install like apps.

A Skill can include:

- Tools — new capabilities the agent can use (e.g., run Playwright tests)
- Instructions — how to use those tools correctly
- Hooks — auto-run commands at key events (pre-commit, post-edit)
- MCP Server configs — pre-configured connections to external services
- Custom agents — specialized personas for specific tasks

Skills are like apps for your AI OS. Install once, use everywhere.

SKILLS vs MCP

Why Skills Are Better Than Raw MCP

	Raw MCP Server	Skill (bundled)
Setup	Manual config, run a server process, manage connections	Install from Extensions view, works immediately
Includes instructions?	No — agent must figure out how to use the tools	Yes — comes with context, rules, and best practices
Portable?	Tied to a running server process	Portable across projects and teams
Composable?	Each server is independent, no orchestration	Skills combine tools + hooks + instructions as one unit
Team sharing	Everyone configures their own MCP setup	Share via repo, install once, everyone has same capabilities

MCP gives you raw power. Skills give you packaged, reusable, shareable intelligence.

PLAYWRIGHT SKILL

Teaching Copilot to See the Browser

Playwright is a browser automation framework. As a Copilot skill, it lets AI control the browser like a human would.

Navigate pages

Open URLs, click links, fill forms

Take screenshots

Capture page state for verification

Assert content

Verify text, elements, layout

Run test flows

Login → create task → verify → delete

Cross-browser

Chrome, Firefox, Safari, mobile viewports

Instruction-driven

"Test the login flow" → AI writes + runs the test

Playwright + Copilot = AI that can test your app like a real user.

COMPUTER USE

AI as the User

AI doesn’t just write code anymore. It uses the computer.

- Clicks buttons
- Fills forms
- Navigates pages
- Takes screenshots and analyzes them
- Verifies UI state visually
- Reports bugs with evidence

INSTRUCTION-DRIVEN TESTING

Write What to Test, AI Executes It

You don’t write test scripts. You write test intentions.

You write:

```
1. Open the To-Do app at localhost:4200
2. Add a new task with title "Buy groceries"
3. Verify the task appears in the list
4. Mark the task as complete
5. Verify it shows with a strikethrough
6. Click the "Completed" filter
7. Verify only completed tasks are shown
8. Delete the task and verify it's gone
```

AI reads these instructions, controls the browser, executes each step, and reports pass/fail with screenshots.

HANDS-ON TIME

Experience the AI Operating System

EXERCISE 1

Create AGENT.md + copilot-instructions.md

What to do:

Use the comprehensive onboarding prompt to create production-grade instruction files for the Banking API. Copilot will inventory the codebase, validate build commands, and document everything.

Try this prompt:

```
Onboard this repository by creating AGENT.md and
.github/copilot-instructions.md. Inventory the
codebase: README, scripts, build files, configs.
For each build/test/run command, validate it works.
Document errors, workarounds, and key file locations.
Keep it under 2 pages. Be specific, not generic.
```

Pro Tip: After creating these files, start a new chat session. Notice how Copilot's responses immediately improve — it knows your project now.

EXERCISE 2

"How Can I Contribute?"

What to do:

Ask Copilot for a personalized contribution guide based on your actual role. This proves that the AGENT.md and instructions files are working.

Try this prompt:

```
Read AGENT.md and copilot-instructions.md.
I am a [your role: developer / QA engineer /
functional analyst / architect / tech lead].

Based on the current state of this project,
give me 5 specific things I can do to contribute.
Prioritize by impact. Be concrete — tell me
exactly which files to look at.
```

Pro Tip: Compare answers across roles in the room. A developer gets code tasks. A QA engineer gets test tasks. Same project, personalized work.

EXERCISE 3

Copilot CLI as Your OS

What to do:

Experience Copilot CLI as an operating system. Do everything from natural language in the terminal — no manual commands.

Try this prompt:

```
Try these in Copilot CLI one at a time:

1. "Show me the project structure"
2. "Run the tests and tell me what passed/failed"
3. "Find all TODO comments in the codebase"
4. "Generate a Dockerfile for this Spring Boot app"
5. "Create a shell script that builds, tests, and
   starts the app in one command"
6. "Commit all changes with a descriptive message"
```

Pro Tip: You just did 6 different OS-level tasks without typing a single traditional command. That's the paradigm shift.

EXERCISE 4

Playwright: Instruction-Driven Testing

What to do:

Set up the Playwright skill and run your first instruction-driven UI test against the To-Do app (or the Banking API if Swagger is available).

Try this prompt:

```
Set up Playwright and test the To-Do app:

1. Open localhost:4200 in the browser
2. Add a task with title "Test from AI"
3. Verify the task appears in the list
4. Mark it as complete
5. Verify it shows as completed
6. Delete it and verify it’s gone

Take a screenshot after each step as evidence.
```

Pro Tip: You just wrote a complete UAT in plain English. No Selenium scripts. No test framework config. Just instructions.

Copilot CLI is the New Developer OS

It reads, writes, executes, orchestrates, and self-corrects.
The terminal is no longer where you type commands.
It’s where you have conversations with your machine.

Reflection

- How did AGENT.md change Copilot’s response quality?
- Which CLI task felt most like using an OS?
- How would instruction-driven testing change your QA workflow?

UP NEXT — DAY 5

Governance & Frameworks